

INFO-F-302, Cours d'Informatique Fondamentale

Emmanuel Filiot et Jean-François Raskin

Département d'Informatique

Faculté des Sciences

Université Libre de Bruxelles

Année académique 2026-2027

Logique *Booléenne* Propositionnelle : Introduction

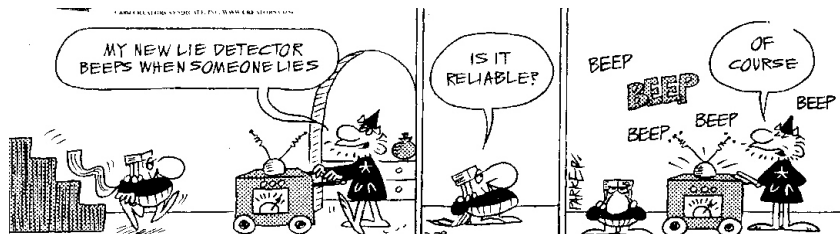
3 Qu'est-ce qu'un langage logique ?

- ▶ langage permettant de décrire avec précision et rigueur des énoncés et des raisonnements, basés notamment sur des connecteurs Booléens (et, ou, négation)
- ▶ langages équipés d'une sémantique claire et non-ambigüe
- ▶ les énoncés logiques seront appelés "formules"
- ▶ *but* : pallier à l'imprécision des langages naturels
- ▶ *but supplémentaire en informatique* : analyser de manière algorithmique les énoncés

4 Fausses vérités et paradoxes

- ▶ l'utilisation du langage naturel comme notation est imprécise et peut mener à des énoncés faux ...
 1. Tout ce qui est rare est cher
 2. Or une chose pas chère, c'est rare
 3. Donc une chose pas chère, c'est cher.
- ▶ ... ou à des paradoxes
 - ▶ Cette phrase est fausse. Problème de *l'auto référence*.
 - ▶ Le barbier rase tous les hommes qui ne se rasent pas eux-mêmes et seulement ceux-là. Qui rase le barbier ?

5 Un petit dernier ...



Exemples d'application de la logique en informatique

- ▶ design de circuits numériques
- ▶ vérification *hardware* et *software*, *model-checking*. Logiques temporelles.
- ▶ preuves de programmes
- ▶ bases de données : SQL.
- ▶ ...

7 Exemple de raisonnement

Considérons la situation décrite par les affirmations suivantes :

1. **Si** le train arrive en retard **et** il n'y a pas de taxis à la gare **alors** l'invité arrive en retard.
2. L'invité n'est pas en retard.
3. Le train est arrivé en retard.

Et la déduction suivante : il y avait des taxis à la gare.

7 Exemple de raisonnement

Considérons la situation décrite par les affirmations suivantes :

1. **Si** le train arrive en retard **et** il n'y a pas de taxis à la gare **alors** l'invité arrive en retard.
2. L'invité n'est pas en retard.
3. Le train est arrivé en retard.

Et la déduction suivante : il y avait des taxis à la gare.

Question

Pourquoi peut-on déduire qu'il y avait des taxis à la gare ?

Premièrement, si on met l'affirmation 1 et l'affirmation 3 ensemble, on peut affirmer que s'il n'y avait pas eu de taxis à la gare, alors l'invité serait arrivé en retard. D'après l'affirmation 2, l'invité n'est pas arrivé en retard. Donc il y avait des taxis à la gare.

8 Autre Exemple

Considérons un autre exemple :

1. **Si** il pleut **et** l'invité a oublié son parapluie **alors** l'invité est trempé.
2. L'invité n'est pas trempé.
3. Il pleut.

Et la déduction suivante : l'invité n'a pas oublié son parapluie.

8 Autre Exemple

Considérons un autre exemple :

1. **Si** il pleut **et** l'invité a oublié son parapluie **alors** l'invité est trempé.
2. L'invité n'est pas trempé.
3. Il pleut.

Et la déduction suivante : l'invité n'a pas oublié son parapluie.

Question

Pourquoi peut-on déduire que l'invité n'a pas oublié son parapluie ?

Premièrement, si on met l'affirmation 1 et l'affirmation 3 ensemble, on peut affirmer que si l'invité avait oublié son parapluie, alors il serait trempé. D'après l'affirmation 2, l'invité n'est pas trempé. Donc l'invité n'a pas oublié son parapluie.

9 Remarque

La deuxième démonstration suit la même **structure logique** que la première démonstration. Il suffit de remplacer les fragments de phrase

	Exemple du train	Exemple du parapluie
comme suit :	Le train est arrivé en retard Il y a des taxis à la gare L'invité est en retard	Il pleut L'invité a son parapluie L'invité est mouillé.

10 Formalisation Logique

Exemple du train	Exemple du parapluie	Proposition
Le train est arrivé en retard	Il pleut	p
Il y a des taxis à la gare	L'invité a son parapluie	q
L'invité est en retard	L'invité est mouillé	r

Démonstration

1. Hypothèse : si p et non q , alors r
2. Hypothèse : p
3. Hypothèse : non r
4. Dédution : si non q alors r
5. Dédution : comme non r , alors q .

11 Formalisation logique

Nous pouvons *formaliser* les trois hypothèses de chacun de ces deux premiers exemples par la *formule logique* suivante :

$$((p \wedge \neg q) \rightarrow r) \wedge \neg r \wedge p$$

Et nous pouvons *formaliser* la déduction par :

$$((p \wedge \neg q) \rightarrow r) \wedge \neg r \wedge p \models q$$

Où “ $\models$ ” se lit : “ q est une conséquence logique de $((p \wedge \neg q) \rightarrow r) \wedge \neg r \wedge p$ ”.

12 Conditions Booléennes dans les langages de programmation

Considérons le test suivant :

if $(x \geq 3 \text{ and } z \leq 3)$ **or** $(y \leq 2 \text{ and } z \leq 3)$ **then** ...

Il peut-être remplacé par

if $(x \geq 3 \text{ or } y \leq 2)$ **and** $z \leq 3$ **then** ...

Car $(p \vee q) \wedge r$ est logiquement équivalent à $(p \wedge r) \vee (q \wedge r)$.

Logique Propositionnelle

Définitions de base : Syntaxe, Sémantique et Exemples

14 Construction des formules

Le *vocabulaire du langage de la logique propositionnelle* est composé :

- ▶ d'un ensemble, fini ou dénombrable, de *propositions* notées $x, y, z, \dots$
Dans la suite, nous notons les ensembles de propositions par les lettres $X, Y, \dots$;
- ▶ de deux constantes : vrai (notée $\top$) et faux (notée $\perp$), parfois notée 1 et 0 ;
- ▶ d'un ensemble de *connecteurs logiques* : et (noté $\wedge$), ou (noté $\vee$), non (noté $\neg$), implique (noté $\rightarrow$), équivalent (noté $\leftrightarrow$);
- ▶ les parenthèses : $(,)$.

15 Construction des formules

Soit X un ensemble de propositions. Les *formules de la logique propositionnelle* respectent la règle de formation BNF (Bachus-Naur Form) suivante :

$$\phi ::= \top \mid \perp \mid x \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \phi_1 \rightarrow \phi_2 \mid \phi_1 \leftrightarrow \phi_2 \mid \neg \phi_1 \mid (\phi_1)$$

Où $\top$, $\perp$ sont respectivement les constantes vrai et faux, $x \in X$ est une proposition et ϕ_1 , ϕ_2 sont des formules propositionnelles bien formées.

La BNF se lit “une formule de la logique propositionnelle est soit la formule $\top$, soit la formule $\perp$, soit une proposition x , soit la conjonction de deux formules ϕ_1 et ϕ_2 (définition inductive), etc.”

16 Quelques exemples de formules

Voici quelques exemples de formules et leur lecture intuitive :

- ▶ la formule propositionnelle " $x \rightarrow (y \wedge z)$ ", peut être lue de la façon suivante " x implique y et z ", ou peut être également lue comme "si x est vrai alors y et z doivent être vrais" ;
- ▶ la formule propositionnelle " $\neg(x \wedge y)$ ", peut-être lue de la façon suivante "il est faux que x et y soient vrais (en même temps)".

17 Ambiguïtés

L'utilisation de la *notation infixe* s'accompagne de problèmes de lecture :

Comment lire $x \wedge y \vee z$? $x \rightarrow y \rightarrow z$?

17 Ambiguïtés

L'utilisation de la *notation infixe* s'accompagne de problèmes de lecture :

Comment lire $x \wedge y \vee z$? $x \rightarrow y \rightarrow z$?

Pour lever les ambiguïtés, on utilise les parenthèses ou des règles de priorité entre opérateurs :

- ▶ si op_1 a une plus grande précedence (priorité) que op_2 alors
 $e_1 \ op_1 \ e_2 \ op_2 \ e_3$ est équivalent à $((e_1 \ op_1 \ e_2)op_2 \ e_3)$
 - ▶ Par ex : $2 \times 3 + 5 = (2 \times 3) + 5 = 11 \neq 2 \times (3 + 5) = 16$.
- ▶ si op_2 a une plus grande précedence (priorité) que op_1 alors
 $e_1 \ op_1 \ e_2 \ op_2 \ e_3$ est équivalent à $(e_1 op_1 (e_2 op_2 e_3))$
 - ▶ Par ex : $2 + 3 \times 5 = 2 + (3 \times 5) = 17$.
- ▶ si op est associatif à gauche alors
 $e_1 \ op \ e_2 \ op \ e_3$ est équivalent à $(e_1 \ op \ e_2)op \ e_3$
 - ▶ Par ex : $10/2/5 = (10/2)/5 = 1 \neq 10/(2/5) = 25$.
- ▶ associativité à droite : $e_1 \ op \ e_2 \ op \ e_3$ se lit $e_1 op(e_2 op e_3)$

On fixe des règles de priorité entre opérateurs afin d'avoir une lecture unique de la formule.

18 Règles de précedence en logique propositionnelle

Ordre de précedence $\prec$ sur les opérateurs :

$$\leftrightarrow \prec \neg \rightarrow \prec \vee \prec \wedge \prec \neg$$

et associativité à gauche pour $\leftrightarrow, \vee, \wedge$ et à droite pour $\rightarrow$.

Exemples

$x \vee y \wedge z$	se lit	$x \vee (y \wedge z)$
$x \rightarrow y \rightarrow x$	se lit	$x \rightarrow (y \rightarrow x)$
$x \vee y \rightarrow z$	se lit	$(x \vee y) \rightarrow z$
$\neg x \wedge y$	se lit	$(\neg x) \wedge y$
$x \rightarrow y \wedge z \rightarrow t$	se lit	$x \rightarrow ((y \wedge z) \rightarrow t)$

18 Règles de précedence en logique propositionnelle

Ordre de précedence $\prec$ sur les opérateurs :

$$\leftrightarrow \prec \rightarrow \prec \vee \prec \wedge \prec \neg$$

et associativité à gauche pour $\leftrightarrow, \vee, \wedge$ et à droite pour $\rightarrow$.

Exemples

$x \vee y \wedge z$	se lit
$x \rightarrow y \rightarrow x$	se lit
$x \vee y \rightarrow z$	se lit
$\neg x \wedge y$	se lit
$x \rightarrow y \wedge z \rightarrow t$	se lit

Remarque

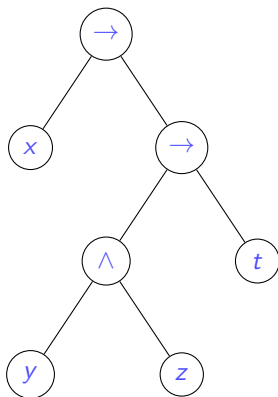
Les parenthèses permettent de contrecarrer ces règles, si elles ne conviennent pas. Elles permettent aussi de rendre une formule plus lisible, ou de ne pas devoir retenir les règles de précedence.

19 Arbre correspondant à une formule

La formule $x \rightarrow y \wedge z \rightarrow t$ est donc équivalente à

$$x \rightarrow ((y \wedge z) \rightarrow t)$$

et donc son arbre de lecture est :



20 La sémantique

- ▶ jusqu'ici, on a vu la syntaxe des formules, qui ne sont rien d'autre que des suites de caractères sans signification.
- ▶ la sémantique donne du sens aux formules, elle permet de les interpréter et de leur attribuer une *valeur de vérité* vrai ou faux.
- ▶ l'interprétation des symboles de proposition est cependant libre et c'est à nous de la fixer. On se donnera donc une *fonction d'interprétation* V pour l'ensemble X de propositions considérées :

$$V : X \rightarrow \{0, 1\}$$

Cette fonction assigne à chaque proposition de X la valeur vrai ou la valeur faux.

- ▶ Dans la suite, nous utiliserons parfois le terme *valuation*, souvent utilisé dans la littérature, au lieu de fonction d'interprétation.
- ▶ A partir d'une fonction d'interprétation V des symboles de proposition, nous allons voir comment interpréter les formules.

21 La sémantique

Définition

- ▶ La *valeur de vérité* d'une formule propositionnelle ϕ formée à partir des propositions d'un ensemble X , évaluée avec la fonction d'interprétation V , est notée $\llbracket \phi \rrbracket_V$. En particulier, on a $\llbracket \phi \rrbracket_V \in \{0, 1\}$.
- ▶ La fonction $\llbracket \phi \rrbracket_V$ est définie par induction sur la syntaxe de ϕ de la façon suivante :
 - ▶ $\llbracket \top \rrbracket_V = 1$; $\llbracket \perp \rrbracket_V = 0$; $\llbracket x \rrbracket_V = V(x)$.
 - ▶ $\llbracket \neg \phi \rrbracket_V = 1 - \llbracket \phi \rrbracket_V$
 - ▶ $\llbracket \phi_1 \vee \phi_2 \rrbracket_V = \max(\llbracket \phi_1 \rrbracket_V, \llbracket \phi_2 \rrbracket_V)$
 - ▶ $\llbracket \phi_1 \wedge \phi_2 \rrbracket_V = \min(\llbracket \phi_1 \rrbracket_V, \llbracket \phi_2 \rrbracket_V)$
 - ▶ $\llbracket \phi_1 \rightarrow \phi_2 \rrbracket_V = \max(1 - \llbracket \phi_1 \rrbracket_V, \llbracket \phi_2 \rrbracket_V)$.
 - ▶ $\llbracket \phi_1 \leftrightarrow \phi_2 \rrbracket_V = \min(\llbracket \phi_1 \rightarrow \phi_2 \rrbracket_V, \llbracket \phi_2 \rightarrow \phi_1 \rrbracket_V)$.

Nous notons $V \models \phi$ si et seulement si $\llbracket \phi \rrbracket_V = 1$, qui se lit “ V satisfait ϕ ”.

22 La sémantique sous forme de tables de vérités

L'information contenue dans la définition précédente est souvent présentée sous forme de tables, appelées *tables de vérité* :

$\top$	$\perp$
1	0

ϕ	$\neg\phi$
1	0
0	1

23 La sémantique sous forme de tables de vérités

ϕ_1	ϕ_2	$\phi_1 \wedge \phi_2$	$\phi_1 \vee \phi_2$	$\phi_1 \rightarrow \phi_2$	$\phi_1 \leftrightarrow \phi_2$
1	1	1	1	1	1
1	0	0	1	0	0
0	1	0	1	1	0
0	0	0	0	1	1

24 Exemples

- Prenons l'interprétation $V_1(x) = 0$, alors on a :

$$\begin{aligned} \llbracket x \rrbracket_{V_1} &= 0 & \llbracket \neg x \rrbracket_{V_1} &= 1 & \llbracket x \wedge x \rrbracket_{V_1} &= 0 & \llbracket x \wedge \neg x \rrbracket_{V_1} &= 0 \\ \llbracket x \rightarrow x \rrbracket_{V_1} &= 1 & \llbracket x \leftrightarrow x \rrbracket_{V_1} &= 1 & \llbracket \neg x \rightarrow x \rrbracket_{V_1} &= 0 \end{aligned}$$

- Prenons la formule $\phi = (x \vee y) \wedge (\neg y \vee z)$ et l'interprétation $V_2(x) = V_2(z) = 1$ et $V_2(y) = 0$. Pour calculer la valeur de vérité de ϕ , on calcule d'abord les valeurs de vérité des sous-formules et on applique les tables de vérité.

$$\begin{aligned} &(x \vee y) \wedge (\neg y \vee z) \\ &(1 \vee 0) \wedge (\neg 0 \vee 1) \\ &(1 \vee 0) \wedge (1 \vee 1) \\ &1 \wedge (1 \vee 1) \\ &1 \wedge 1 \\ &1 \end{aligned}$$

Donc $\llbracket \phi \rrbracket_{V_2} = 1$.

25 Exemples

- Prenons la même formule $\phi = (x \vee y) \wedge (\neg y \vee z)$ et l'interprétation $V_3(z) = 1$ et $V_3(x) = V_3(y) = 0$.

$$\begin{aligned} & (x \vee y) \wedge (\neg y \vee z) \\ & (0 \vee 0) \wedge (\neg 0 \vee 1) \\ & (0 \vee 0) \wedge (1 \vee 1) \\ & 0 \wedge (1 \vee 1) \\ & 0 \wedge 1 \\ & 0 \end{aligned}$$

Donc $\llbracket \phi \rrbracket_{V_3} = 0$.

26 Exemples

On peut utiliser le principe de décomposition en sous-formule et l'application des tables de vérité pour obtenir la valeur de vérité de ϕ pour toutes les interprétations possibles des variables

x	y	z	$(x \vee y)$	$\neg y$	$(\neg y \vee z)$	$(x \vee y) \wedge (\neg y \vee z)$
1	1	1				
1	1	0				
1	0	1				
1	0	0				
0	1	1				
0	1	0				
0	0	1				
0	0	0				

26 Exemples

On peut utiliser le principe de décomposition en sous-formule et l'application des tables de vérité pour obtenir la valeur de vérité de ϕ pour toutes les interprétations possibles des variables

x	y	z	$(x \vee y)$	$\neg y$	$(\neg y \vee z)$	$(x \vee y) \wedge (\neg y \vee z)$
1	1	1	1			
1	1	0	1			
1	0	1	1			
1	0	0	1			
0	1	1	1			
0	1	0	1			
0	0	1	0			
0	0	0	0			

26 Exemples

On peut utiliser le principe de décomposition en sous-formule et l'application des tables de vérité pour obtenir la valeur de vérité de ϕ pour toutes les interprétations possibles des variables

x	y	z	$(x \vee y)$	$\neg y$	$(\neg y \vee z)$	$(x \vee y) \wedge (\neg y \vee z)$
1	1	1	1	0		
1	1	0	1	0		
1	0	1	1	1		
1	0	0	1	1		
0	1	1	1	0		
0	1	0	1	0		
0	0	1	0	1		
0	0	0	0	1		

26 Exemples

On peut utiliser le principe de décomposition en sous-formule et l'application des tables de vérité pour obtenir la valeur de vérité de ϕ pour toutes les interprétations possibles des variables

x	y	z	$(x \vee y)$	$\neg y$	$(\neg y \vee z)$	$(x \vee y) \wedge (\neg y \vee z)$
1	1	1	1	0	1	
1	1	0	1	0	0	
1	0	1	1	1	1	
1	0	0	1	1	1	
0	1	1	1	0	1	
0	1	0	1	0	0	
0	0	1	0	1	1	
0	0	0	0	1	1	

26 Exemples

On peut utiliser le principe de décomposition en sous-formule et l'application des tables de vérité pour obtenir la valeur de vérité de ϕ pour toutes les interprétations possibles des variables

x	y	z	$(x \vee y)$	$\neg y$	$(\neg y \vee z)$	$(x \vee y) \wedge (\neg y \vee z)$
1	1	1	1	0	1	1
1	1	0	1	0	0	0
1	0	1	1	1	1	1
1	0	0	1	1	1	1
0	1	1	1	0	1	1
0	1	0	1	0	0	0
0	0	1	0	1	1	0
0	0	0	0	1	1	0

27 Exemples

- $\neg x \vee y$ (qui se lit $(\neg x) \vee y$)

x	y	$\neg x$	$\neg x \vee y$
1	1	0	1
1	0	0	0
0	1	1	1
0	0	1	1

- Remarque : c'est la même table de vérité que pour l'implication :

x	y	$x \rightarrow y$
1	1	1
1	0	0
0	1	1
0	0	1

- On dira que les deux formules $\neg x \vee y$ et $x \rightarrow y$ sont **équivalentes**.

28 Remarques sur l'implication

- ▶ l'implication $\phi_1 \rightarrow \phi_2$ modélise le raisonnement *si-alors* :
Si ϕ_1 est vraie, **ALORS** ϕ_2 est vraie aussi.
- ▶ le cas où ϕ_1 est faux ne nous intéresse pas dans l'implication.
- ▶ par conséquent, si ϕ_1 est faux, alors peu importe la valeur de vérité de ϕ_2 , l'implication $\phi_1 \rightarrow \phi_2$ reste vraie.

29 Un dernier exemple : $\phi = \neg(x \wedge y) \leftrightarrow (\neg x \vee \neg y)$

- Calculons la table de vérité :

x	y	$x \wedge y$	$\neg(x \wedge y)$	$\neg x$	$\neg y$	$\neg x \vee \neg y$	ϕ
1	1	1	0	0	0	0	1
1	0	0	1	0	1	1	1
0	1	0	1	1	0	1	1
0	0	0	1	1	1	1	1

- pour toutes les interprétations possibles des propositions, ϕ est vraie. On dira dans ce cas que ϕ est **valide**.
- En fait, on a montré ici que les deux formules $\neg(x \wedge y)$ et $\neg x \vee \neg y$ sont équivalentes. C'est une des lois de Morgan.

30 Exercices

Dire pour les interprétations V et les formules ϕ suivantes si $V \models \phi$:

1. $V(x) = 1$, $V(y) = 0$ et $\phi = (x \rightarrow y) \wedge (x \rightarrow \neg y)$
2. $V(x) = V(y) = 0$ et $\phi = ((\neg x \rightarrow y) \rightarrow (\neg y \rightarrow x)) \wedge (x \vee y)$
3. $V(x) = 0$ et $V(y) = 1$ et $\phi = ((\neg x \vee y) \rightarrow (y \wedge (x \leftrightarrow y)))$
4. $V(x) = V(z) = 1$ et $V(y) = 0$ et
 $\phi = ((\neg z \rightarrow \neg x \wedge \neg y) \vee t) \leftrightarrow (x \vee y \rightarrow z \vee t)$
5. $V(x) = V(y) = 0$ et $V(z) = 1$ et
 $\phi = (x \wedge (y \rightarrow z)) \leftrightarrow ((\neg x \vee y) \rightarrow (x \wedge z)).$

$$\begin{array}{ccccccc}
 0 & 0 & 1 & 1 & 0 & 0 & 1 \\
 \underbrace{}_0 & \leftrightarrow & \underbrace{(1 \rightarrow 0)}_1 & \underbrace{}_0
 \end{array}$$

$$\begin{array}{ccc}
 0 & \leftrightarrow & 0 \\
 \underbrace{}_1
 \end{array}$$

$$\llbracket \phi \rrbracket_V = 1$$

$V(x) = V(z) = 1$ et $V(y) = 0$ et

$$\phi = ((\neg z \rightarrow \neg x \wedge \neg y) \vee t) \leftrightarrow (x \vee y \rightarrow z \vee t)$$

0

0

1

1

0

1

0 $\rightarrow$ 0

1 $\rightarrow$ 1

1

1

1 $\vee$ t

1

1

$\leftrightarrow$

1

1

$$\llbracket \phi \rrbracket_V = 1$$

31 Validité et Satisfaisabilité

Voici deux définitions importantes :

Définition (Satisfaisabilité)

Une formule propositionnelle ϕ est *satisfaisable* si et seulement si il existe une fonction d'interprétation V pour les propositions de ϕ , telle que $V \models \phi$.

Définition (Validité)

Une formule propositionnelle ϕ est *valide* si et seulement si pour toute fonction d'interprétation V pour les propositions de ϕ , on a $V \models \phi$.

32 Exemples

formule	satisfaisable	valide
$\top$	oui	oui
x	oui $V(x) = 1$	non $V(x) = 0$
$\neg x$	oui $V(x) = 0$	non $V(x) = 1$
$(x \vee y) \wedge (\neg y \vee z)$	oui voir table précédente	non voir table précédente
$x \wedge \neg x$	non	non
$x \wedge \neg y \wedge (\neg y \rightarrow \neg x)$	non	non
$x \vee \neg x$	oui	oui
$(x \rightarrow y) \vee (y \rightarrow x)$	oui	oui

33 Notion de conséquence logique

Définition

Soit $\phi_1, \dots, \phi_n, \phi$ des formules. On dira que ϕ est une *conséquence logique* de $\phi_1, \dots, \phi_n$, noté $\phi_1, \dots, \phi_n \models \phi$, si $(\phi_1 \wedge \dots \wedge \phi_n) \rightarrow \phi$ est valide.

Par exemple, $p, \neg p \models \perp$, et $\phi_1, \phi_1 \rightarrow \phi_2 \models \phi_2$.

34 Application importante de la notion de validité

Définition

Deux formules ϕ et ψ sont dites *équivalentes* si la formule $\phi \leftrightarrow \psi$ est valide. On notera $\phi \equiv \psi$ pour signifier que ϕ et ψ sont équivalentes.

- ▶ dans une formule, on peut substituer une sous-formule par une autre équivalente sans changer la sémantique de la formule de départ
- ▶ par exemple, dans la formule

$$\gamma = (x \vee y) \wedge \neg(x \wedge z)$$

on peut remplacer la sous-formule $\neg(x \wedge z)$ par $(\neg x \vee \neg z)$ et on obtient la formule suivante, qui est équivalente à γ :

$$\gamma' = (x \vee y) \wedge (\neg x \vee \neg z)$$

- ▶ l'application d'équivalences simples va nous permettre de simplifier des formules, et de les mettre sous des formes particulières

35 Quelques équivalences

Pour toutes formules ϕ_1, ϕ_2, ϕ_3 , on a :

- ▶ $\neg\neg\phi_1 \equiv \phi_1$ (double négation)
- ▶ $\neg(\phi_1 \wedge \phi_2) \equiv (\neg\phi_1 \vee \neg\phi_2)$ (loi de De Morgan pour le 'et')
- ▶ $\neg(\phi_1 \vee \phi_2) \equiv (\neg\phi_1 \wedge \neg\phi_2)$ (loi de De Morgan pour le 'ou')
- ▶ $\phi_1 \wedge (\phi_2 \vee \phi_3) \equiv (\phi_1 \wedge \phi_2) \vee (\phi_1 \wedge \phi_3)$ (distributivité du 'et')
- ▶ $\phi_1 \vee (\phi_2 \wedge \phi_3) \equiv (\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3)$ (distributivité du 'ou')
- ▶ $\phi_1 \rightarrow \phi_2 \equiv \neg\phi_2 \rightarrow \neg\phi_1$ (contraposition)

36 Comment démontrer ces équivalences ? (1)

Prenons par exemple l'équivalence suivante :

$$\phi_1 \vee (\phi_2 \wedge \phi_3) \equiv (\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3)$$

On doit démontrer que la formule

$(\phi_1 \vee (\phi_2 \wedge \phi_3)) \leftrightarrow ((\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3))$ est valide, c'est-à-dire que quelque soit la valeur de vérité des formules ϕ_1, ϕ_2, ϕ_3 , les deux formules à droite et à gauche de l'implication ont la même sémantique. Pour cela, on pourrait faire une table de vérité :

ϕ_1	ϕ_2	ϕ_3	$\phi_2 \wedge \phi_3$	$\phi_1 \vee \phi_2$	$\phi_1 \vee \phi_3$	$\phi_1 \vee (\phi_2 \wedge \phi_3)$	$(\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3)$
1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	0	0	0	1	1	1	1
0	1	1	1	1	1	1	1
0	1	0	0	1	0	0	0
0	0	1	0	0	1	0	0
0	0	0	0	0	0	0	0

37 Comment démontrer ces équivalences ? (2)

On peut aller plus vite que faire la table de vérité. Nous devons démontrer que pour toute interprétation V , nous avons que $V \models \phi_1 \vee (\phi_2 \wedge \phi_3)$ si et seulement si $V \models (\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3)$. Nous considérons deux cas :

1. si $V \models \phi_1$, alors on obtient par définition de la disjonction :
 $V \models \phi_1 \vee (\phi_2 \wedge \phi_3)$, $V \models (\phi_1 \vee \phi_2)$, et $V \models (\phi_1 \vee \phi_3)$. Donc
 $V \models (\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3)$.
2. si $V \not\models \phi_1$. Alors $V \models \phi_1 \vee (\phi_2 \wedge \phi_3)$ ssi $V \models \phi_2 \wedge \phi_3$ ssi $V \models \phi_2$ et $V \models \phi_3$, ssi $V \models \phi_1 \vee \phi_2$ et $V \models \phi_1 \vee \phi_3$, ssi
 $V \models (\phi_1 \vee \phi_2) \wedge (\phi_1 \vee \phi_3)$.

38 Négation de l'implication

Quand est-ce que la négation d'une implication $A \rightarrow B$ est vraie ? Dès lors que l'hypothèse (A) est vraie, mais pas la conclusion (B). Autrement dit, lorsqu'on a $A \wedge \neg B$.

On peut le démontrer formellement avec les règles d'équivalences :

$$\neg(A \rightarrow B) \equiv \neg(\neg A \vee B) \equiv \neg\neg A \wedge \neg B \equiv A \wedge \neg B$$

39 Lien entre satisfaisabilité et validité

Théorème

Une formule propositionnelle ϕ est valide ssi sa négation $\neg\phi$ n'est pas satisfaisable.

Par conséquent, si on dispose d'un algorithme qui décide si une formule est satisfaisable ou non, on obtient un algorithme qui décide si la formule est valide, il suffit de tester la (non-)satisfaisabilité de sa négation.

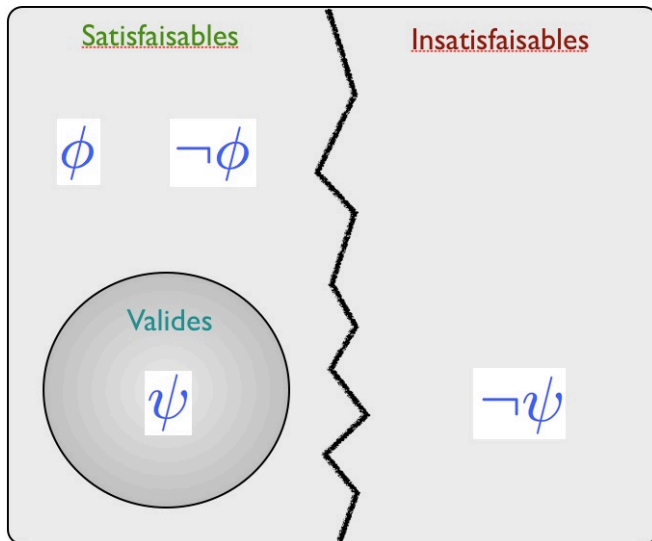
Remarque : ce théorème permet de réduire le problème de validité (décider si oui ou non une formule est valide) en un problème d'insatisfaisabilité (décider si oui ou non une formule est insatisfaisable). On peut d'ailleurs concevoir un algorithme qui fait cette réduction : il doit simplement prendre la négation de la formule.

40 Démonstration du théorème

Nous devons démontrer une équivalence (un si et seulement si). On peut donc supposer ce qui est à gauche de l'équivalence et en déduire ce qui est à droite, et réciproquement.

1. supposons que ϕ est valide. Alors pour tout interprétation V , on a $V \models \phi$. Donc $V \not\models \neg\phi$. Comme c'est pour toute interprétation quelconque V , cela signifie que $\neg\phi$ n'est pas satisfaisable.
2. réciproquement, supposons que $\neg\phi$ ne soit pas satisfaisable. Alors pour tout interprétation V quelconque, $V \not\models \neg\phi$, donc $V \models \phi$. De nouveau, comme c'est pour tout interprétation quelconque V , on en déduit que ϕ est valide.

41 Diagramme



42 Comment tester la satisfaisabilité d'une formule ?

42 Comment tester la satisfaisabilité d'une formule ?

- ▶ on a vu la méthode des tables de vérité : tester toutes les interprétations de propositions jusqu'à ce qu'on en trouve une qui satisfait la formule
- ▶ **problème** : quelle est la complexité d'un tel algorithme ? combien d'interprétations faut-il tester si la formule contient n propositions ?

42 Comment tester la satisfaisabilité d'une formule ?

- ▶ on a vu la méthode des tables de vérité : tester toutes les interprétations de propositions jusqu'à ce qu'on en trouve une qui satisfait la formule
- ▶ **problème** : quelle est la complexité d'un tel algorithme ? combien d'interprétations faut-il tester si la formule contient n propositions ?
- ▶ il faut essayer, dans le pire des cas (c'est à dire le cas où la formule n'est pas satisfaisable), 2^n interprétations : cet algorithme a donc une complexité exponentielle dans le nombre de propositions
- ▶ ce n'est pas raisonnable pour les applications que nous allons aborder, car nous allons générer des formules contenant plusieurs centaines de propositions.
- ▶ nous allons maintenant étudier un algorithme "plus intelligent" : la méthode des tableaux sémantiques.

43 Tableaux sémantiques

C'est un algorithme pour établir la satisfaisabilité de formules de la logique propositionnelle.

On a besoin d'une nouvelle définition :

Définition

Un littéral est une proposition x ou la négation d'une proposition $\neg x$.

44 Tableaux sémantiques : exemple

Considérons la formule $\phi = x \wedge (\neg y \vee \neg x)$

Essayons de construire systématiquement une fonction d'interprétation V telle que

$$\llbracket \phi \rrbracket_V = 1$$

Par définition on a

$$\llbracket \phi \rrbracket_V = 1$$

$$\llbracket x \rrbracket_V = 1 \quad \text{ssi} \quad \text{et} \quad \llbracket \neg y \vee \neg x \rrbracket_V = 1$$

et

$$\begin{aligned} & \llbracket \neg y \vee \neg x \rrbracket_V = 1 \\ & \quad \text{ssi} \\ & \llbracket \neg y \rrbracket_V = 1 \quad \text{ou} \quad \llbracket \neg x \rrbracket_V = 1 \end{aligned}$$

et donc,

$$\begin{aligned} & \llbracket \phi \rrbracket_V = 1 \\ & \quad \text{ssi} \\ & \llbracket x \rrbracket_V = 1 \quad \text{et} \quad \llbracket \neg y \rrbracket_V = 1 \\ & \text{ou} \quad \llbracket x \rrbracket_V = 1 \quad \text{et} \quad \llbracket \neg x \rrbracket_V = 1 \end{aligned}$$

Pour ϕ , on construit la fonction d'interprétation V de la façon suivante :

- ▶ $V(x) = 1$;
- ▶ $V(y) = 0$.

et on a bien que $V \models \phi$.

46 Tableaux sémantiques

- ▶ Un ensemble S de littéraux est satisfaisable ssi il ne contient pas une paire de *littéraux complémentaires*. Par exemple, $\{x, \neg y\}$ est satisfaisable alors que $\{x, \neg x\}$ ne l'est pas.
- ▶ Nous avons réduit le problème de satisfaction de ϕ à un problème de satisfaction d'ensembles de littéraux : ϕ est satisfaisable ssi $\{x, \neg y\}$ est satisfaisable ou $\{x, \neg x\}$ est satisfaisable.

$$(x \vee y) \wedge (\neg x \wedge \neg y) \quad 1 \text{ but}$$

$$x \vee y, \quad \underline{\neg x \wedge \neg y} \quad 2 \text{ buts}$$

$$\underline{x \vee y}, \quad \neg x, \neg y$$

$$\underline{x, \neg x}, \neg y$$

feuille
fermée

$$y, \underline{\neg x, \neg y}$$

feuille fermée

Donc la formule de départ est insatisfaisable

$$\neg x \wedge (x \rightarrow y)$$

$$\neg x, \underline{x \rightarrow y}$$

$$\neg x, \neg x$$

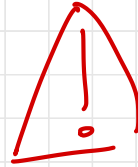
ouvert

$V(x) = 0$ satisfait
la feuille et donc
la formule de départ
aussi

$$\neg x, y$$

ouvert

$V(x) = 0 \quad V(y) = 1$
satisfait la feuille
et donc la formule
aussi.



la formule
n'est pas
valide pourtant
toutes les
feuilles sont
satisfaisables

47 Tableaux sémantiques

Un autre exemple : $\phi = (x \vee y) \wedge (\neg x \wedge \neg y)$.

Par définition, on a

$$\begin{array}{c} \llbracket \phi \rrbracket_v = 1 \\ \text{ssi} \\ \llbracket x \vee y \rrbracket_v = 1 \text{ et } \llbracket \neg x \wedge \neg y \rrbracket_v = 1 \end{array}$$

et donc,

$$\begin{array}{c} \llbracket \phi \rrbracket_v = 1 \\ \text{ssi} \\ \llbracket x \vee y \rrbracket_v = 1 \text{ et } \llbracket \neg x \rrbracket_v = 1 \text{ et } \llbracket \neg y \rrbracket_v = 1 \end{array}$$

et donc,

$$\begin{array}{c} \llbracket \phi \rrbracket_v = 1 \\ \text{ssi} \\ \text{soit } \llbracket x \rrbracket_v = 1 \text{ et } \llbracket \neg x \rrbracket_v = 1 \text{ et } \llbracket \neg y \rrbracket_v = 1 \\ \text{ou } \llbracket y \rrbracket_v = 1 \text{ et } \llbracket \neg x \rrbracket_v = 1 \text{ et } \llbracket \neg y \rrbracket_v = 1 \end{array}$$

48 Tableaux sémantiques

et donc,

ϕ est satisfaisable

ssi

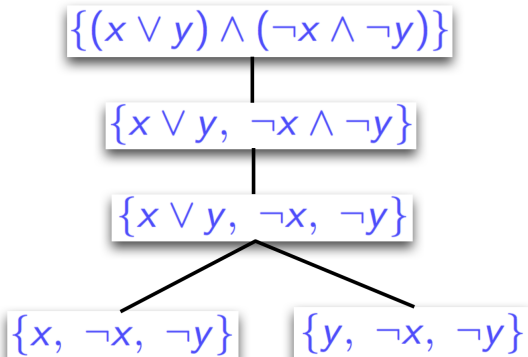
$\{x, \neg x, \neg y\}$ est satisfaisable

ou $\{y, \neg x, \neg y\}$ est satisfaisable.

Vu que les deux ensembles de littéraux contiennent chacun une proposition et sa négation, la formule ϕ n'est pas satisfaisable.

49 Tableaux sémantiques

L'information présente dans les développements précédents est plus facilement représentée sous forme d'un arbre :



50 Tableaux sémantiques – Remarques

- ▶ quand on applique le "pas de simplification" à un noeud où on sélectionne une conjonction, alors on obtient un seul fils; on appelle ces règles, des $\wedge$ -règles;
- ▶ quand on applique le "pas de simplification" à un noeud où on sélectionne une disjonction, alors on obtient deux fils; on appelle ces règles, des $\vee$ -règles.

51 Règles de simplification : $\wedge$ -règles

Etant donné un ensemble S contenant une formule α , on crée un ensemble fils égal $S \setminus \{\alpha\}$ auquel on ajoute α_1 et α_2 .

α	α_1	α_2
$\neg\neg\phi$	ϕ	
$\phi_1 \wedge \phi_2$	ϕ_1	ϕ_2
$\neg(\phi_1 \vee \phi_2)$	$\neg\phi_1$	$\neg\phi_2$
$\neg(\phi_1 \rightarrow \phi_2)$	ϕ_1	$\neg\phi_2$
$\phi_1 \leftrightarrow \phi_2$	$\phi_1 \rightarrow \phi_2$	$\phi_2 \rightarrow \phi_1$

$\wedge$ -règles .

Remarque : toutes les formules α peuvent être considérées comme équivalentes à des conjonctions. Par exemple :

$\neg(\phi_1 \vee \phi_2)$ est équivalent à $\neg\phi_1 \wedge \neg\phi_2$

52 Règles de simplification : $\vee$ -règles

Etant donné un ensemble S contenant une formule β , on crée deux ensembles fils, l'un étant $(S \cup \{\beta_1\}) \setminus \beta$, et l'autre $(S \cup \{\beta_2\}) \setminus \beta$.

β	β_1	β_2
$\phi_1 \vee \phi_2$	ϕ_1	ϕ_2
$\neg(\phi_1 \wedge \phi_2)$	$\neg\phi_1$	$\neg\phi_2$
$\phi_1 \rightarrow \phi_2$	$\neg\phi_1$	ϕ_2
$\neg(\phi_1 \leftrightarrow \phi_2)$	$\neg(\phi_1 \rightarrow \phi_2)$	$\neg(\phi_2 \rightarrow \phi_1)$

$\vee$ -règles .

Remarque : toutes les formules β peuvent être considérées comme équivalentes à des disjonctions. Par exemple :

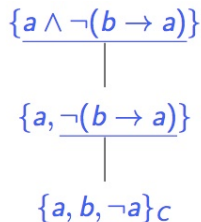
$$\neg(\phi_1 \wedge \phi_2) \text{ est équivalent à } \neg\phi_1 \vee \neg\phi_2$$

53 Algorithme

L'algorithme construit à partir d'une formule ϕ un arbre noté T_ϕ dont les noeuds sont des ensembles de formules.

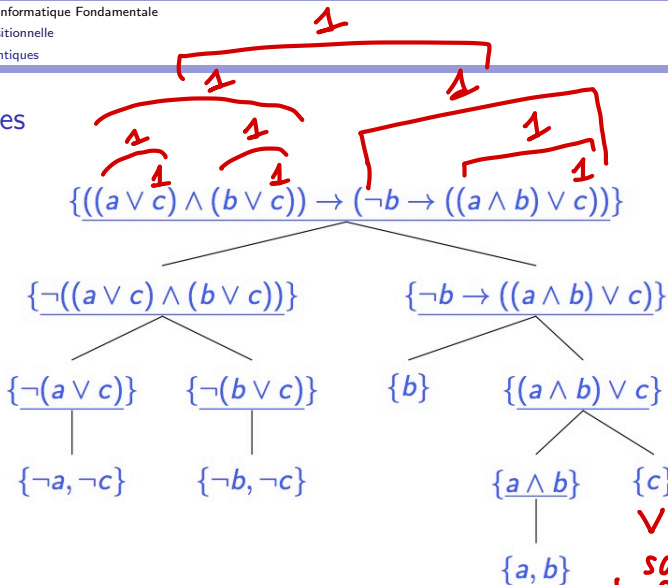
- ▶ **Initialisation** : au départ, l'arbre ne contient qu'un seul noeud : l'ensemble $\{\phi\}$;
- ▶ **Tant qu'**il existe une feuille de l'arbre F qui contient une formule ψ qui est simplifiable
 - ▶ **si** ϕ est simplifiable par une $\wedge$ -règle, alors on crée un fils F' à F où F' est obtenu supprimant ϕ de F et en ajoutant la formule (dans le cas d'une double négation) ou les deux formules (pour les autres cas) obtenues par la simplification, à l'ensemble F'
 - ▶ **si** ϕ est simplifiable par une $\vee$ -règle, alors on crée deux fils F_1 et F_2 , chacun étant obtenu en supprimant ϕ de F et en ajoutant respectivement les formules obtenues par la simplification.
- ▶ **si** toutes les feuilles de l'arbre contiennent une paire de littéraux complémentaires, **alors retourner** non-satisfaisable, **sinon retourner** satisfaisable.

54 Exemples



Toutes les feuilles (ici une seule) contiennent une paire de littéraux complémentaires (a et $\neg a$), donc la formule n'est pas satisfaisable.

55 Exemples



Il existe plusieurs feuilles ne contenant pas de paire de littéraux complémentaires, la formule est donc satisfaisable.

$V(c) = 1$
satisfait
la formule
quelque
soit les
valeurs de b et a

56 Questions

Soit ϕ une formule et T_ϕ un arbre construit avec l'algorithme précédent.

1. Si toutes les feuilles de T_ϕ sont satisfaisables, est-ce que cela signifie que ϕ est valide ?

56 Questions

Soit ϕ une formule et T_ϕ un arbre construit avec l'algorithme précédent.

1. Si toutes les feuilles de T_ϕ sont satisfaisables, est-ce que cela signifie que ϕ est valide? Non : prendre par exemple $\phi = x$.
2. quelle est la taille maximale de T_ϕ (nombre de noeuds) en fonction du nombre de symboles de ϕ ?

56 Questions

Soit ϕ une formule et T_ϕ un arbre construit avec l'algorithme précédent.

1. Si toutes les feuilles de T_ϕ sont satisfaisables, est-ce que cela signifie que ϕ est valide? Non : prendre par exemple $\phi = x$.
2. quelle est la taille maximale de T_ϕ (nombre de noeuds) en fonction du nombre de symboles de ϕ ? T_ϕ peut avoir un nombre exponentiel de noeuds. Par exemple, prenons $n \geq 0$ et $\phi = (x_1 \vee y_1) \wedge (x_2 \vee y_2) \cdots \wedge (x_n \vee y_n)$, alors T_ϕ a au moins $2^{n+1} + n - 2$ noeuds, et exactement 2^n feuilles. L'algorithme a donc une complexité exponentielle en temps dans le pire cas.

$$\vdash \underbrace{(x_1 \vee y_1)} \wedge \underbrace{(x_2 \vee y_2)} \cdots \wedge \underbrace{(x_n \vee y_n)}, \varepsilon$$

|
⋮
|

$$\{ \underline{x_1 \vee y_1}, x_2 \vee y_2, \dots, x_n \vee y_n \}$$

$$\{ x_1, \underline{x_2 \vee y_2}, \dots, x_n \vee y_n \}$$

$$\{ y_1, \underline{x_2 \vee y_2}, \dots, x_n \vee y_n \}$$

$$\{ x_1, x_2, x_3 \vee y_3, \dots \}$$

$$\{ x_1, y_2, x_3 \vee y_3, \dots \}$$

$$\{ y_1, x_2, \dots \}$$

$$\{ y_1, y_2, \dots \}$$



⋮

⋮

⋮

⋮

57 Tableaux Sémantiques : Résumé

- ▶ la méthode des tableaux sémantiques est un algorithme pour tester la satisfaisabilité d'une formule.
- ▶ elle est basée sur la construction d'un arbre par applications successives de règles de simplifications de formules (tableaux)
- ▶ les formules équivalentes à une disjonction (disjonction, implication, négation d'une conjonction, négation d'une équivalence) sont satisfaites si un des deux membres de la disjonction est satisfait : les deux choix sont représentés par du branchement (deux fils) dans l'arbre.
- ▶ les formules équivalentes à une conjonction (conjonction, équivalence, négation d'une implication et d'une disjonction) sont satisfaites si les deux membres de la conjonction sont satisfaits : un seul fils est créé dans l'arbre.
- ▶ les feuilles de l'arbre sont des ensembles de littéraux (donc non simplifiables), pour lesquels la satisfaisabilité est facile à tester (absence de littéraux complémentaires).
- ▶ dans le pire cas, l'arbre créé est exponentiellement plus grand que la formule
- ▶ pour tester la validité d'une formule, on peut tester la non satisfaisabilité de sa négation par la méthode des tableaux sémantiques.

EXERCICE

(examen 2^{de} session 2026)

Démontrer la validité de

$$\alpha = (p \rightarrow (q \vee r)) \rightarrow ((p \wedge \neg q) \rightarrow r)$$

On sait que α est valide ssi

$\neg \alpha$ n'est pas satisfaisable

→ On applique la méthode
des tableaux sémantiques sur $\neg \alpha$

$$\neg \left((p \rightarrow (q \vee r)) \rightarrow ((p \wedge \neg q) \rightarrow r) \right)$$

$$p \rightarrow (q \vee r), \quad \underline{\neg ((p \wedge \neg q) \rightarrow r)}$$

$$p \rightarrow (q \vee r), \quad \underline{p \wedge \neg q}, \quad \neg r$$

$$\underline{p \rightarrow (q \vee r)}, \quad p, \quad \neg q, \quad \neg r$$

$$\underline{\neg p, p, \neg q, \neg r}$$

la feuille est fermée
(insatisfaisable)

$$\underline{q \vee r}, \quad p, \quad \neg q, \quad \neg r$$

$$\underline{q, p, \neg q, \neg r} \quad \text{fermée}$$

$$\underline{r, p, \neg q, \neg r} \quad \text{fermée}$$

Toutes les feuilles sont fermées, donc $\neg \alpha$ est
insatisfaisable, donc α est valide.